

## Part 1

```
In [3]: import nest_asyncio
nest_asyncio.apply()
```

```
In [ ]: import os
import scrapy
from scrapy.crawler import CrawlerProcess
import uuid

class Crawler(scrapy.Spider):
    name = "web_crawler"

    def __init__(self, seed_url, max_pages, max_depth, save_dir=None, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.start_urls = [seed_url]
        self.allowed_domains = ['en.wikipedia.org']
        self.max_pages = int(max_pages)
        self.pages_crawled = 0
        self.save_dir = save_dir or 'downloaded_pages'
        self.custom_settings = {
            'DEPTH_LIMIT': int(max_depth),
            'DOWNLOAD_DELAY': 1
        }

    def parse(self, response):
        if self.pages_crawled >= self.max_pages:
            return

        os.makedirs(self.save_dir, exist_ok=True)
        filename = f'{str(uuid.uuid4()).upper()}.html'
        filepath = os.path.join(self.save_dir, filename)
        with open(filepath, "wb") as f:
            f.write(response.body)
        self.pages_crawled += 1

        if self.pages_crawled >= self.max_pages:
            print("\n=====")
            print(f"Total pages saved: {self.pages_crawled}")
            print("=====\n")
            return

        for href in response.css('a::attr(href)').getall():
            if href and href.startswith('/wiki/') and not any(x in href for x in ['&', '?']):
                yield response.follow(href, self.parse)

crawl_process = CrawlerProcess(settings = {'LOG_LEVEL': 'INFO'})
crawl_process.crawl(Crawler, seed_url='https://en.wikipedia.org/wiki/Main_Page', max_pages=10)
try:
    crawl_process.start()
except:
    pass
```

```

2025-12-07 21:27:36 [scrapy.utils.log] INFO: Scrapy 2.13.4 started (bot: scrapybot)
2025-12-07 21:27:36 [scrapy.utils.log] INFO: Versions:
{'lxml': '6.0.2',
 'libxml2': '2.11.9',
 'cssselect': '1.3.0',
 'parsel': '1.10.0',
 'w3lib': '2.3.1',
 'Twisted': '25.5.0',
 'Python': '3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 '
          '64 bit (AMD64)]',
 'pyOpenSSL': '25.3.0 (OpenSSL 3.5.4 30 Sep 2025)',
 'cryptography': '46.0.3',
 'Platform': 'Windows-11-10.0.26200-SP0'}
2025-12-07 21:27:36 [scrapy.addons] INFO: Enabled addons:
[]
2025-12-07 21:27:36 [scrapy.extensions.telnet] INFO: Telnet Password: bfed19f4db5620
0b
2025-12-07 21:27:36 [scrapy.middleware] INFO: Enabled extensions:
['scrapy.extensions.corestats.CoreStats',
 'scrapy.extensions.telnet.TelnetConsole',
 'scrapy.extensions.logstats.LogStats']
2025-12-07 21:27:36 [scrapy.crawler] INFO: Overridden settings:
{'LOG_LEVEL': 'INFO'}
2025-12-07 21:27:36 [scrapy.middleware] INFO: Enabled downloader middlewares:
['scrapy.downloadermiddlewares.offsite.OffsiteMiddleware',
 'scrapy.downloadermiddlewares.httpauth.HttpAuthMiddleware',
 'scrapy.downloadermiddlewares.downloadtimeout.DownloadTimeoutMiddleware',
 'scrapy.downloadermiddlewares.defaultheaders.DefaultHeadersMiddleware',
 'scrapy.downloadermiddlewares.useragent.UserAgentMiddleware',
 'scrapy.downloadermiddlewares.retry.RetryMiddleware',
 'scrapy.downloadermiddlewares.redirect.MetaRefreshMiddleware',
 'scrapy.downloadermiddlewares.httpcompression.HttpCompressionMiddleware',
 'scrapy.downloadermiddlewares.redirect.RedirectMiddleware',
 'scrapy.downloadermiddlewares.cookies.CookiesMiddleware',
 'scrapy.downloadermiddlewares.httpproxy.HttpProxyMiddleware',
 'scrapy.downloadermiddlewares.stats.DownloaderStats']
2025-12-07 21:27:36 [scrapy.middleware] INFO: Enabled spider middlewares:
['scrapy.spidermiddlewares.start.StartSpiderMiddleware',
 'scrapy.spidermiddlewares.httperror.HttpErrorMiddleware',
 'scrapy.spidermiddlewares.referer.RefererMiddleware',
 'scrapy.spidermiddlewares.urllength.UrlLengthMiddleware',
 'scrapy.spidermiddlewares.depth.DepthMiddleware']
2025-12-07 21:27:36 [scrapy.middleware] INFO: Enabled item pipelines:
[]
2025-12-07 21:27:36 [scrapy.core.engine] INFO: Spider opened
2025-12-07 21:27:36 [scrapy.extensions.logstats] INFO: Crawled 0 pages (at 0 pages/m
in), scraped 0 items (at 0 items/min)
2025-12-07 21:27:36 [scrapy.extensions.telnet] INFO: Telnet console listening on 12
7.0.0.1:6023

```

```

=====
Total pages saved: 10
=====

```

```

2025-12-07 21:28:36 [scrapy.extensions.logstats] INFO: Crawled 1071 pages (at 1071 p
ages/min), scraped 0 items (at 0 items/min)
2025-12-07 21:29:23 [scrapy.core.engine] INFO: Closing spider (finished)
2025-12-07 21:29:23 [scrapy.statscollectors] INFO: Dumping Scrapy stats:
{'downloader/request_bytes': 1619821,
 'downloader/request_count': 3238,
 'downloader/request_method_count/GET': 3238,
 'downloader/response_bytes': 199863153,
 'downloader/response_count': 3238,
 'downloader/response_status_count/200': 3238,
 'dupefilter/filtered': 1317,
 'elapsed_time_seconds': 107.375166,
 'finish_reason': 'finished',
 'finish_time': datetime.datetime(2025, 12, 8, 3, 29, 23, 716059, tzinfo=datetime.ti
mezone.utc),
 'httpcompression/response_bytes': 1053330599,
 'httpcompression/response_count': 3238,
 'items_per_minute': 0.0,
 'log_count/INFO': 14,
 'request_depth_max': 2,
 'response_received_count': 3238,
 'responses_per_minute': 1815.700934579439,
 'scheduler/dequeued': 3238,
 'scheduler/dequeued/memory': 3238,
 'scheduler/enqueued': 3238,
 'scheduler/enqueued/memory': 3238,
 'start_time': datetime.datetime(2025, 12, 8, 3, 27, 36, 340893, tzinfo=datetime.tim
ezone.utc)}
2025-12-07 21:29:23 [scrapy.core.engine] INFO: Spider closed (finished)

```

## Part 2.1

```

In [5]: from bs4 import BeautifulSoup
import re, os

def build_inverted_index(directory: str, output_json: str):
    inverted_index = {}
    documents = []
    document_ids = []

    for filename in os.listdir(directory):
        if not filename.endswith('.html'):
            continue
        doc_id = os.path.splitext(filename)[0]
        file_path = os.path.join(directory, filename)
        with open(file_path, 'r', errors='ignore') as f:
            html = f.read()

        soup = BeautifulSoup(html, 'lxml')
        text = soup.get_text(' ', strip=True)

        documents.append(text)
        document_ids.append(doc_id)

        tokens = re.findall(r"\b\w+\b", text.lower())

```

```

for position, token in enumerate(tokens):
    if token not in inverted_index:
        inverted_index[token] = []
    if inverted_index[token] and inverted_index[token][-1][0] == doc_id:
        inverted_index[token][-1][1].append(position)
    else:
        inverted_index[token].append((doc_id, [position]))

with open('index.json', 'w', encoding='utf-8') as f:
    f.write('index = {\n')
    items = list(inverted_index.items())
    for i, (token, entries) in enumerate(items):
        f.write(f' "{token}": [\n')
        for j, (doc_id, positions) in enumerate(entries):
            doc_str = f"{doc_id}"
            pos_str = "[" + ", ".join(map(str, positions)) + "]"
            line = f"          ({doc_str}, {pos_str})"
            if j < len(entries) - 1:
                line += ","
            f.write(line + "\n")

        if i < len(items) - 1:
            f.write(' ],\n')
        else:
            f.write(' ]\n')
    f.write('}\n')

print(f"inverted index saved to {output_json}")
print(f"Total unique terms in inverted index: {len(inverted_index)}")

build_inverted_index('downloaded_pages', 'index.json')

```

inverted index saved to index.json

Total unique terms in inverted index: 11378

## Part 2.2

In [6]: `from sklearn.feature_extraction.text import TfidfVectorizer`  
`from sklearn.metrics.pairwise import cosine_similarity`

```

def build_text(folder_name):
    documents = {}
    path = os.path.join(os.getcwd(), folder_name)
    for filename in os.listdir(path):
        if not filename.lower().endswith('.html'):
            continue
        doc_id = os.path.splitext(filename)[0]
        file_path = os.path.join(path, filename)
        with open(file_path, 'r', encoding='utf-8') as f:
            html = f.read()
        soup = BeautifulSoup(html, 'html.parser')
        for tag in soup(['script', 'style']):
            tag.decompose()
        text = soup.get_text(separator=' ')
        text = ' '.join(text.split())
        documents[doc_id] = text

```

```

    return documents

def search(query, documents):
    doc_ids = list(documents.keys())
    corpus = [documents[doc_id] for doc_id in doc_ids]

    vectorizer = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
    tfidf_matrix = vectorizer.fit_transform(corpus)
    query_vector = vectorizer.transform([query])
    cosine_scores = cosine_similarity(query_vector, tfidf_matrix).flatten()

    results = []

    for idx, doc_id in enumerate(doc_ids):
        row = tfidf_matrix[idx].toarray()[0]

        tfidf_terms = {}
        for token in query.lower().split():
            if token in vectorizer.vocabulary_:
                term_index = vectorizer.vocabulary_[token]
                tfidf_weight = float(row[term_index])
            else:
                tfidf_weight = 0.0
            tfidf_terms[token] = tfidf_weight
        term_str = ", ".join([f"{term}: {value:.4f}" for term, value in tfidf_terms.items()])
        results.append({
            'doc_id': doc_id,
            'cosine': float(cosine_scores[idx]),
            'tfidf_str': term_str
        })

    results.sort(key=lambda x: x["cosine"], reverse=True)

    return results

query = 'Computer science and mathematics'
documents = build_text('downloaded_pages')
result = search(query, documents)

for r in result[:6]:
    print(f'Document {r['doc_id']}: ')
    print(f'    Cosine Similarity Score: {r['cosine']:.4f}')
    print(f'    TF-IDF Weights: {r['tfidf_str']}\n')

```

Document BBBE3CEC-CE7D-43F8-B9D2-975B0AA2D505:

Cosine Similarity Score: 0.0748

TF-IDF Weights: computer: 0.0000, science: 0.0088, and: 0.2381, mathematics: 0.0000

Document 269EE64D-E4BA-4B62-8074-DAAC4700D2D1:

Cosine Similarity Score: 0.0747

TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.2535, mathematics: 0.0000

Document 1444005F-E4DD-494B-9CFD-64D0095CA1AA:

Cosine Similarity Score: 0.0701

TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.2378, mathematics: 0.0000

Document A841512B-67E0-4C33-991B-4514FC9E92C4:

Cosine Similarity Score: 0.0701

TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.2378, mathematics: 0.0000

Document 9A48B2B1-9AD9-41FD-B212-76320A84C579:

Cosine Similarity Score: 0.0641

TF-IDF Weights: computer: 0.0000, science: 0.0233, and: 0.1758, mathematics: 0.0000

Document 55DB4099-1BE1-41DF-96C6-991D5EFBEDB3:

Cosine Similarity Score: 0.0635

TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.2156, mathematics: 0.0000

### Part 3

```
In [7]: import threading
from flask import Flask, request, send_file, jsonify
import pandas as pd
import io
import requests

documents = build_text('downloaded_pages')

app = Flask(__name__)

@app.route("/process", methods=["POST"])
def processor():
    try:
        if 'file' not in request.files:
            return jsonify({'error': 'No file was uploaded'}), 400

        filename = request.files['file']
        top_k = int(request.form.get('top_k', 3))

        try:
            df = pd.read_csv(filename, dtype=str)
        except Exception as error:
```

```

        return jsonify({'error': f'CSV read error: {str(error)}'}), 400
    required = {'query_id', 'query_text'}
    if not required.issubset(df.columns):
        return jsonify({'error': 'CSV must contain query_id and query_text'}),

    output_rows = []

    for _, row in df.iterrows():
        qid = row.get('query_id', '').strip()
        qtext = row.get('query_text', '').strip()
        if not qid:
            return jsonify({'error': 'query_id cannot be empty'}), 400
        if not qtext:
            return jsonify({"error": f"Empty query_text for query_id {qid}"})
        try:
            ranked = search(qtext, documents)
        except Exception as e:
            return jsonify({
                'error': f'Search failed for query_id {qid}: {str(e)}'}, 500
        for rank, item in enumerate(ranked[:top_k], start=1):
            output_rows.append({
                'query_id': qid,
                'rank': rank,
                'document_id': item['doc_id']
            })
    out_df = pd.DataFrame(output_rows)

    out_buffer = io.StringIO()
    out_df.to_csv(out_buffer, index=False)
    out_buffer.seek(0)

    return send_file(
        io.BytesIO(out_buffer.getvalue().encode("utf-8")),
        mimetype="text/csv",
        as_attachment=True,
        download_name='results2.csv'
    )

except Exception as fatal:
    return jsonify({'error': f'Unexpected server error: {str(fatal)}'}), 500

def run_app():
    app.run(port=5000)
thread = threading.Thread(target=run_app)
thread.start()

url = 'http://127.0.0.1:5000/process'
files = {'file': open('queries.csv', 'rb')}
data = {'top_k': 3}
response = requests.post(url, files=files, data=data)

if not response.ok:
    print('Error: ', response.text)
else:

```

```
with open('results.csv', 'wb') as f:
    f.write(response.content)

print(f'Output saved to results.csv with top-{data['top_k']} ranked retrieval')
```

```
* Serving Flask app '__main__'
```

```
* Debug mode: off
```

```
2025-12-07 21:28:12 [werkzeug] INFO: WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
```

```
* Running on http://127.0.0.1:5000
```

```
2025-12-07 21:28:12 [werkzeug] INFO: Press CTRL+C to quit
```

```
2025-12-07 21:28:13 [werkzeug] INFO: 127.0.0.1 - - [07/Dec/2025 21:28:13] "POST /process HTTP/1.1" 200 -
```

```
Output saved to results.csv with top-3 ranked retrieval
```

## Abstract

This project includes three main parts: a web crawler, an inverted index builder and searching algorithm, and a processing algorithm for Top-K ranked retrieval. The development process involved an incremental approach, where each part of the project was developed and completed before moving on to the next part. Additionally, initial testing was also done before moving to other parts of the project to ensure a previously developed function works correctly. Once all parts were completed, more internal testing (not reflected on GitHub repository) was done to ensure all parts worked correctly, especially with large inputs.

Throughout the development process, dedicated days to debugging and understanding a part of the project were allocated to ensure a smoother and more efficient development. If more time were to be dedicated to this project, the next steps in development would be adding query spelling-correction and suggestions, vector embedding representation, and distributed crawling. This would improve search accuracy by correcting typos and offering alternative terms, enable retrieval of contextually similar documents, and enhance scalability through large-scale, continuous updates to the web crawler.

## Overview

The solution outline is as follows:

- Web Crawler - Used a scrapy crawler with CrawlerProcess; starts at a base URL and recursively follows links up to a specified depth and amount of .html files need
- Indexer - Used BeautifulSoup to extract the text from the folder, tokenized each term within the folder, and built an positional inverted index as a .json file. The search algorithm calculates the cosine similarity and the TF-IDF weights of each term in comparison to the corpus for a specified query.
- Processor - A Flask based processor with Top-K ranked retrieval that validates query text and query IDs within the queries.csv file and outputs a results.csv file containing query ID, rank, and document ID.

Based on this solution outline, the following literatures were used to gain information about possible implementation methods:

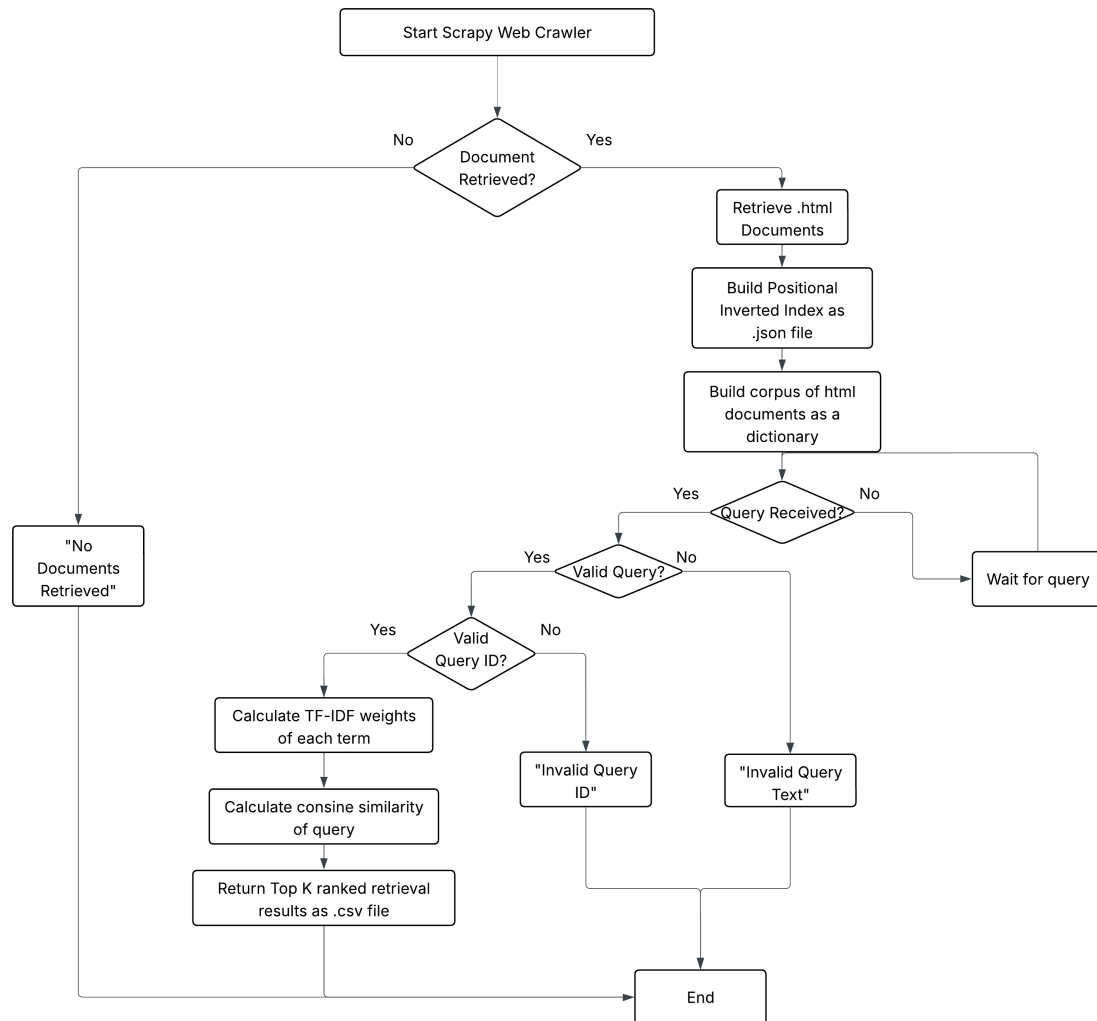
- [Scrapy Documentation](#): Used to learn about a scrapy crawler through CrawlerProcess. Provided useful information on relevant custom settings and functions that can be used to create a web crawler.
- [Flask Documentation](#): Used to learn about creating a Flask application and setting up a GET request for the project's processor algorithm. It also provided information on using send file to save a csv file into a user's directory.
- [Sklern Documentation](#): Used to learn more about calculating TF-IDF weights and cosine similarity scores.

The system will use Scrapy based web crawler to gather html documents and store them in a folder. This folder will later be retrieved by a function that builds an positional inverted index and stores the index in a json file. A query will be given to a search function that calculate its cosine similarity to each document and TF-IDF weights, relative to the corpus. These calculation will then be used to display Top-K ranked retrieved documents after each query ID and query text is processed and error checked

## Design

Based on a specified depth and max pages, the system will be able to scrap through a specific website and gather a set of html documents into a folder. The greater the depth specified, the deeper the crawler will go through each link found in each html document. Based on these results, the folder of html files is used to create a positional inverted index with document IDs, terms and positions in which each term exists in a document. Furthermore, the folder of html files is converted into a corpus with a python dictionary and used to build a search algorithm. This search algorithm accepts a query and calculates its cosine similarity and TF-IDF weights of each term within the query, relative to each document. These results are used within a processing algorithm that provides error checking and query validation, and Top-K ranked retrieval. This processing function uses the search algoirhtm on a queries.csv file, which contains query text and query IDs and provides the Top-K ranked retrieval results through a results.csv file containing document IDs, query IDs, and ranks for each document.

The following diagram was created to showcase the interactions between each project component:



In terms of integration, the web crawler goes through a specified website with a user determined depth and max pages, and stores the result in a folder. Within the folder, it contains .html files that were scrapped from the specified website. This document folder is then used to build the positional inverted index to show the positions of each term in the documents and their respective positions. The same document folder is built as a corpus using a python dictionary for the basis of the search and processor algorithms. A valid query text and query ID are expected from the user through a csv file, with proper text validations and error checking before the search algorithm is used on each query. Once the search algorithm is called, it uses the previously built corpus to calculate the cosine similarity of the query and TF-IDF weights of each term in the query, relative to each document. Based on these results, the processor algorithm will output a specified Top-K ranked retrieval (predetermined to be 4) as a csv file.

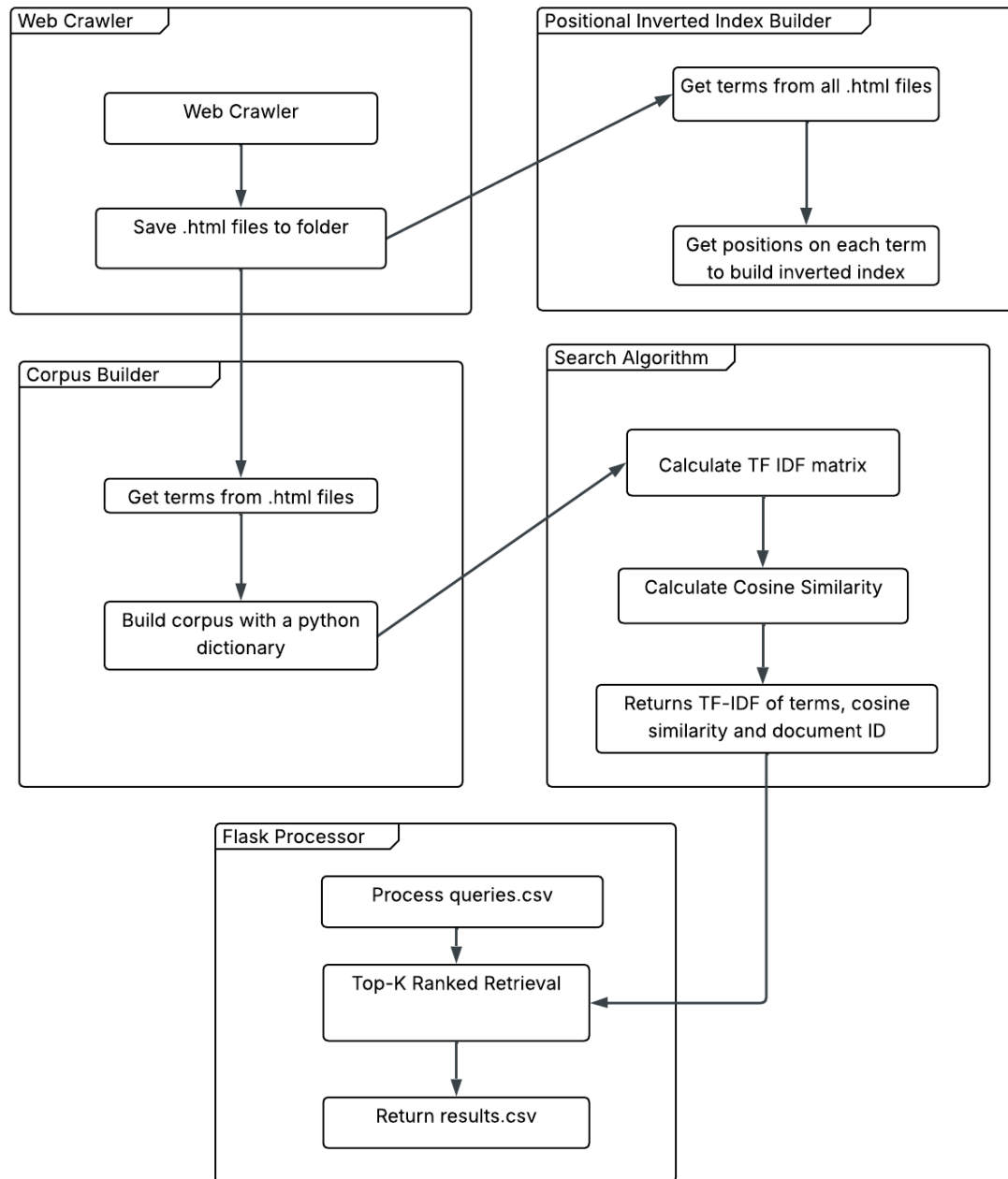
## Architecture

The software components of this project are the following:

- Scrapy based Web Crawler

- Positional Inverted Index Builder, corpus builder and search algorithm
- Processor algorithm with error checking and query validation, with Top-K ranked retrieval

The following diagram was created to explain the communication:



The web crawler was built using the CrawlerProcess library, with custom settings for depth limit and a one-second download delay. These settings were applied to adhere to the safe crawling rules that wikipedia has in place, while also preserving the speed of the crawler. Within the constructor of the Crawler class, a seed\_url, maximum depth, maximum pages, and a directory are provided by the user. In addition, the constructor sets the base domain to "en.wikipedia.org." This is done to avoid the crawler going into wikipedia pages that

contain another language. With the name of the new directory is given, the OS library was used to create a new folder that would store the results from the web crawler as .html files. The UUID library was used to create the document IDs for the .html files. Within the parsing function in the Crawler class, the response from the scrapy crawler would be written into each .html file, which is specified through the allowed max pages. Furthermore, a check was added to ensure any web pages that were crawled had the tag "/wiki/" within their link. This tag notifies the web crawler to only scrap wikipedia articles, avoiding an special web pages, categories, and files. After these checks are performed, the crawler uses the response.follow() function to follow the specified link. In order to run the web crawler with crawl\_process.start(), the nest\_asyncio.apply() function from the nest\_asyncio library was used so it can run smoothly. Additionally, an argument of settings = {"LOG\_LEVEL": "INFO"} was passed through the CrawlerProcess() function to only display important logging information and avoid unnecessary GET 200 requests from displaying.

To build the positional inverted index, the function used the "BeautifulSoup" library to extract terms the .html files. A text document was constructed for each .html file and the results where stored in a list, alongside their respective document IDs where the text was extracted from. The library "re" was used to properly tokenize each term and ignores all punctuation. The inverted index is then built with the tokens, document IDs, and the positions of each term in a given document. These results, stored in a list of tuples, are written into a .json file and formatted as the professor's example through custum string formatting. The search algorithm uses a helper function ("build text") that builds a proper corpus by extracting the directory with the .html files. After this extraction, the library "Beautiful Soup" is used to extract text from the .html files and connect them with their respective document IDs. Based on these results, its passed as an argument, alongside a query, to compute the TF-IDF matrix and cosine similarity. Using Scikit learn, TfidfVectorizer and cosine similarity are imported to calculate each respective value. The library TfidfVectorizer is uses to remove any punctuation and to label the importance of each term in a document. These results are used to create the TF IDF matrix through the "fit transform" function. A query vector is created through the function "transform." Based on these calculation, the cosine similarity is calculated with the query vector and TF-IDF matrix as arguments. The function then returns the document ID, consine similarity, and the TF-IDF weights for the specified query terms. Within this implementation, the "build text" was necessary to build the corpus as the "fit transform" function only accepts raw text documents; this is the main reason why the positional inverted index is not used to create the TF-IDF matrix.

A Flask processor function was created with a POST route with the top k and the queries.csv file being sent through the post() function. The queries.csv file goes through error checking pipeline to validate if a file was sent through the post() function. For the top k value, it is not required to be an argument for the post() function since a default value of 3 is set for top k ranked retrieval. When the file is successfully retrieved, the data within the csv file is extracted using pandas library, specifically through a dataframe for easier access. Once the

csv file is extracted using `pd.read_csv()` function, the processor function iterates through the dataframe to extract query ID and query text. Once the extractions are successful, both fields are checked if they are valid (ensuring neither field is empty). Once this is completed, the search function is used to pass the query text and the corpus (created using `build_text()` function). With the results from the search algorithm, a new csv file is created ("results.csv") that store the document ID, query text, and rank. Before writing the results into the results.csv, the results are ranked based on a specified top-k value. For the purposes of this report, the default top-k value of 3 was used. The ranked results are stored in a list ("out-df"), which is then converted into a dataframe using the `pd.DataFrame()` function. To begin the writing process, a buffer is created to tell the `to_csv()` function to have the data be converted into strings. The `to_csv()` function is used on the "out-df" dataframe and is returned as a csv by the processing function using `send_file()`. The processor function returns results.csv file with ranked retrieval of  $k = 3$  and displaying the ranks between each document ID and query ID. In the final submission, the k value is set to 10 for top 10 ranked retrieval. Other error checking done throughout the processor function was unexpected server errors, failure of the `search()` algorithm on a specific query id, and csv read errors. Since the development of the Flask processor was in a jupyter notebook, a thread was created to safely run the Flask application. When running the application without a thread, the cell never finished and kernel kept running until interrupted. Hence, creating a thread with `threading.Thread()` and `thread.start()` was the safest option for safe execution.

## Operation

To run the set up the environment, the following packages need to be installed:

- Scrapy
- Flask
- Scikit Learn
- BeautifulSoup
- nest\_asyncio
- Pandas
- Requests

The following commands can be done to install each package:

- Scrapy: `pip install Scrapy`
- Flask: `pip install flask`
- Scikit Learn: `pip install scikit-learn`

- BeautifulSoup: pip install beautifulsoup4
- nest\_asyncio: pip install nest\_asyncio
- pandas: pip install pandas
- requests: pip install requests

Since the codebase is based in a jupyter notebook, no additional commands are needed to run the environment or the code.

The following inputs are given for each major part of the project:

- Scrapy Web Crawler: (seed\_url = [https://en.wikipedia.org/wiki/Main\\_Page](https://en.wikipedia.org/wiki/Main_Page), max\_pages = 10, max\_depth = 1, save\_dir = "downloaded\_pages")
- Inverted Index Builder: ("downloaded\_pages", "index.json") ; The "index.json" input will be the name of the output json file
- Build Text (corpus builder): ("downloaded\_pages")
- Search Algorithm: (query, corpus) ; corpus is retrieved from the build\_text() function
- Flask Processor: (file = "queries.csv", data = "top-k" : 3)

## Conclusion

Overall, I was able to successfully implement a functioning Scrapy web crawler, an index builder, a search algorithm using Scikit Learn, and a processing algorithm using Flask. An iterative approach was used to build all components of the project, where I ensured all previous parts worked before moving on to the next part of the project. Once all parts were completed, I revisited each component to optimize it and make any necessary adjustments based on changes made elsewhere in the project. One example of this was outputting the results from the Search() algorithm where initially, it only displayed TF-IDF weight for each term of the query and the cosine similarity. Later in the approach, I changed the output result so it is more organized and concise with the TF-IDF weight of each term in the query, cosine similarity, and associated document ID. Based on this, the iterative approach I adopted allowed me to effectively split the work of the project to avoid feeling overwhelmed. Additionally, all results from each component of the project were satisfactory and displayed the correct information required. The results.csv file, index.json, the folder with the .html files, and the results from the Search algorithm were all properly generated with no issues. These results can be seen in the GitHub Repository for this project.

However, one of the biggest challenges that I heavily struggled on to find a solution was dealing with "ReactorNotRestartable" exception. This exception did not crash the web crawler or interrupt its functionality. Rather, this exception happened after the web crawler completed its function. I understand this exception happens because crawl\_process.start()

cannot run in the same event loop and environment twice. This exception occurred primarily because of how Jupyter Notebook handles cell execution. Each time a cell is run, Jupyter Notebooks attempts to start Scrapy's underlying Twisted reactor again, which is not allowed since the reactor can only be started once per Python process. I tried other solutions such as `CrawlerRunner()` and `Threads`, but both raised separate issues; `CrawlerRunner()` took too long to execute and `Threads` did not allow me to not display the GET 200 request logging information. Hence, a soft solution I applied was running `nest_asyncio.apply()` from the `nest_asyncio` library. Additionally, I used a try-catch statement to ignore the "ReactorNotRestartable" exception from displaying in the final results.

## Data Sources

The following links were used for the Scrapy web crawler:

- Seed URL: "[https://en.wikipedia.org/wiki/Main\\_Page](https://en.wikipedia.org/wiki/Main_Page)"
- Domain Allowed: "en.wikipedia.org"

For the domain, only english wikipedia articles were allowed to avoid the documents and inverted index from containing terms from other languages. In order to safely crawl through wikipedia and follow wikipedia's crawling guidelines, a download delay of one second was added to the web crawler.

## Test Cases

The following test cases were done for the project:

- Scrapy Web Crawler: a. Maximum depth of one with maximum pages of 10.
  - a. Maximum depth of one with maximum pages of 10 => results shown in the report PDF
  - b. Maximum depth of three with maximum pages of 100 => results shown in Canvas submission and in the Jupyter Notebook in the GitHub Repository.
- Inverted Index Builder
  - a. Maximum depth of one with maximum pages of 10 => results shown in the report PDF
  - b. Maximum depth of three with maximum pages of 100 => results shown in Canvas submission and in the Jupyter Notebook in the GitHub Repository. These results were from a previous iteration so results may differ from GitHub Repository.

Results from a.

```
inverted index saved to index.json  
Total unique terms in inverted index: 11378
```

Results from b.

```
inverted index saved to index.json  
Total unique terms in inverted index: 42183
```

- Search Algorithm

a. query = 'Computer science and mathematics' with maximum depth of one and maximum pages of 10

b. query = 'Computer science and mathematics' with maximum depth of three and maximum pages of 100. These results were from a previous iteration so results may differ from GitHub Repository.

Results from a.

```
Document BBBE3CEC-CE7D-43F8-B9D2-975B0AA2D505:  
Cosine Similarity Score: 0.0748  
TF-IDF Weights: computer: 0.0000, science: 0.0088, and: 0.2381, mathematics: 0.0000  
  
Document 269EE64D-E4BA-4B62-8074-DAAC4700D2D1:  
Cosine Similarity Score: 0.0747  
TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.2535, mathematics: 0.0000  
  
Document 1444005F-E4DD-494B-9CFD-64D0095CA1AA:  
Cosine Similarity Score: 0.0701  
TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.2378, mathematics: 0.0000  
  
Document A841512B-67E0-4C33-991B-4514FC9E92C4:  
Cosine Similarity Score: 0.0701  
TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.2378, mathematics: 0.0000  
  
Document 9A48B2B1-9AD9-41FD-B212-76320A84C579:  
Cosine Similarity Score: 0.0641  
TF-IDF Weights: computer: 0.0000, science: 0.0233, and: 0.1758, mathematics: 0.0000  
  
Document 55DB4099-1BE1-41DF-96C6-991D5EFBEDB3:  
Cosine Similarity Score: 0.0635  
TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.2156, mathematics: 0.0000
```

Results from b.

```

Document 96C9E2BF-6449-4113-9D47-350B9CD72E02:
  Cosine Similarity Score: 0.0797
  TF-IDF Weights: computer: 0.0083, science: 0.0185, and: 0.2478, mathematics: 0.0331

Document BEE06908-53DE-4062-9F9A-DC203E274F37:
  Cosine Similarity Score: 0.0656
  TF-IDF Weights: computer: 0.0087, science: 0.0277, and: 0.2372, mathematics: 0.0090

Document E72DF70B-C7A6-42B8-9C78-F72C7167C907:
  Cosine Similarity Score: 0.0507
  TF-IDF Weights: computer: 0.0000, science: 0.0066, and: 0.2605, mathematics: 0.0000

Document 6D8B8D27-A017-4D9C-ACAC-47D6C2DACFBD:
  Cosine Similarity Score: 0.0507
  TF-IDF Weights: computer: 0.0000, science: 0.0066, and: 0.2602, mathematics: 0.0000

Document 503B4483-A373-43A0-99F2-7D796C7BAF94:
  Cosine Similarity Score: 0.0498
  TF-IDF Weights: computer: 0.0396, science: 0.0000, and: 0.1333, mathematics: 0.0000

Document 72EAB349-DB7A-4816-A528-C3F51D09E373:
  Cosine Similarity Score: 0.0493
  TF-IDF Weights: computer: 0.0000, science: 0.0000, and: 0.1946, mathematics: 0.0202

```

- Processor Function
  - a. queries.csv with top-k = 3 => results shown in the report PDF
  - b. queries.csv with top-k = 10 => results shown in Canvas submission and in the Jupyter Notebook in the GitHub Repository

## Source Code

- [Flask Documentation](#): Used this documentation for general information about several Flask related functions (e.g., POST routes, jsonify, send\_file). This documentation is used in the Flask Processor section of the project ("Part 3")
- [Scrapy Documentation](#): Used this documentation for information regarding how to use CrawlerProcess() and .start() functions. This documentation is used in the web crawler section of the project ("Part 1")
- [Scikit Learn Documentation](#): Used this documentation for general information regarding the implementation on TF-IDF weight and cosine similarity using Scikit Learn. This documentation is used in the Search algorithm section of the project ("Part 2.2").
- [Example on creating a POST API endpoint in Flask](#): Used as a reference point on how to create a POST API endpoint in Flask and how to extract data being sent. This example was useful for the processor function section of the project ("Part 3").
- [Pandas Documentation](#): Used to gather information related to converting csv into a dataframe and iterating through a dataframe. This was used in the Flask processor

section of the project ("Part 3").

- [Beautiful Soup 4 Documentation](#): Used to gather functions and information about parsing .html files from the "downloaded\_pages" folder. This was used in the inverted index builder and corpus builder function ("Part 2.1" and "Part 2.2").

Here are list of dependencies used for this project:

- nest\_asyncio: version 1.6.0
- Scrapy: version 2.13.4
- Flask: versions 3.1.2
- Scikit Learn: version 1.7.2
- BeautifulSoup4: version 4.14.2
- Python: version 3.12.4
- Pandas: version 2.3.3
- Requests: version 2.32.5

## Bibliography

[1] "Core API — Scrapy Documentation." <https://docs.scrapy.org/en/latest/topics/api.html>

[2] "Flask Documentaion." <https://flask.palletsprojects.com/en/stable/api/>

[3] 2025. How to Create a POST API Endpoint in Flask." devCamp.  
<https://devcamp.com/trails/python-api-development-with-flask/campsites/279/guides/how-to-create-post-api-endpoint-flask>

[4] 2025. "Implementing Web Scraping in Python with Scrapy." Sanchhaya Education Private Limited. <https://www.geeksforgeeks.org/python/implementing-web-scraping-python-scrapy/>

[5] Leonard Richardson. 2004-2025. "Beautiful Soup 4 Documentation."  
<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

[6] "OS Library." <https://docs.python.org/3/library/os.html>

[7] 2025. "pandas documentation." <https://pandas.pydata.org/docs/>

[8] 1999–2025. "Pandas Read CSV." Efsnes Data.  
[https://www.w3schools.com/python/pandas/pandas\\_csv.asp](https://www.w3schools.com/python/pandas/pandas_csv.asp)

- [9] 2025. "Python RegEx." Sanchhaya Education Private Limited.  
<https://www.geeksforgeeks.org/python/regular-expression-python-examples/>
- [10] Sakthivel Thangaraj. 2024. "Introduction to nest\_asyncio for Python developer." Medium.  
<https://medium.com/@sakthiveltvt.thangaraj/introduction-to-nest-asyncio-for-python-developers-afd7bed44768>
- [11] "Scikit Learn Documentation." <https://scikit-learn.org/stable/api/sklearn.html>
- [12] 2025. "Understanding TF-IDF (Term Frequency-Inverse Document Frequency)." Sanchhaya Education Private Limited. <https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/>
- [13] 2025. "Use jsonify() instead of json.dumps() in Flask." Sanchhaya Education Private Limited. <https://www.geeksforgeeks.org/python/use-jsonify-instead-of-json-dumps-in-flask/>
- [14] "UUID Objects." <https://docs.python.org/3/library/uuid.html>